

Parallel and Distributed Computing

OpenMP Report

Juan Camilo Rodriguez Viteri Pedro Afonso Saraiva Gomes João Paulo Ferreira Russo
115766 *103468* *116543*

Group 3

1 Introduction

This report covers the parallelization of the serial version for the Documents Classification Problem. Overall, the main goal is to evaluate the performance differences between the sequential implementation and the parallelized version, and focusing on how it was achieved.

The problem at hand consists in assigning a set of documents to cabinets. Each document contains a fixed amount of scores, known as subjects, which are used for the assignment. Cabinets are defined at each iteration by the centroid of their current members, that is, the average score vector of all documents assigned to them. The algorithm starts with a round-robin distribution, then repeatedly recomputes centroids and moves each document to the nearest one until no document changes cabinet.

The parallel implementation that will be presented aims to accelerate the work through threads. Whilst overcoming additional issues which could be propagated by parallelizing, such as synchronization and load balancing.

2 Serial Implementation

Prior to covering the main focus of this report, we briefly want to disclose and cover some improvements that were made between the original serial version and the current version that is being delivered alongside this report.

The first version kept a list of documents per cabinet. To compute centroids, it went through each cabinet's list and averaged the scores. For reassignment, it could not move documents immediately as it found their new cabinet, because that would break the iteration,

so it collected all the moves first and applied them at the end. We had to do two passes per iteration.

The second version removes the per-cabinet document lists. The key point is that a centroid only needs the total scores and the count of documents, not the actual list of who is in the cabinet. So instead of building that list and scanning it later, the score sums are updated right as each document is assigned. By the end of the reassignment loop the centroid data is already complete. One pass per iteration, and no intermediate bookkeeping.

For both implementations, cabinet scores were kept in a single array. This was done so that we could benefit from spatial data locality whenever we process cabinets sequentially. The same was done for document scores.

3 Parallelization Approach

Our implementation uses OpenMP in order to parallelize our serialized version. Prior to elaborating further as to how the application was parallelized, its important to have an understanding at a high-level the different elements of the application. The core of the algorithm is broken in the following functions:

- **assign_to_cabinets** - initial round-robin assignment of documents to cabinets.
- **recalculate_scores** - recalculates the centroid scores for each cabinet.
- **reassign_documents** - reassignment of documents to cabinets.

An important realization from early stages was the need to parallelize as promptly as possible in our code, meaning from the point we would start timing the algorithm execution. Therefore, there is a single `#pragma omp parallel` that encapsulates the algorithm. The threads created in this region are used

throughout the remainder of the execution. There is no invocation of other parallel regions or termination of threads, purposely done in order to avoid unnecessary overhead, and having to destroy and/or create new threads.

Another important OpenMP directive present in our implementation is the `#pragma omp for` directive. Due to the sheer nature of the problem at hand, specifically the quantity of iterative computations, there was a clear need to include this directive in key areas where loops were present in order to perform multiple calculations in parallel, as our HotSpot Analysis made it clear that these were the areas where the program spent most of its time.

4 Work Decomposition

The algorithm has three loops that run repeatedly until convergence: `assign_to_cabinets`, `recalculate_scores`, and `reassign_documents`.

Both loops, `assign_to_cabinets` and `reassign_documents` iterate over all D documents. Each document's work does not depend on any other document in the same iteration, so the loop is split across threads with `#pragma omp for`, in which each thread gets a chunk of the documents. The costlier of the two is the reassignment step, where each document computes its distance to all C cabinets and picks the closest one, giving $O(C \cdot S)$ work per document.

The issue is that assigning a document also writes to the cabinet accumulators. Each assignment adds the document's score vector to `new_scores` and increments `new_counts` for that cabinet. Two threads adding different document scores to the same cabinet would write to the same memory slot, the later thread would have its cache line be invalidated and would have to refetch it to be able to update the score. Even updating scores of different cabinets is an issue, as long as the cabinet score blocks share a cache line, false sharing will happen.

To avoid this, each thread gets its own private block within `new_counts` and `new_scores`. Additionally, blocks from one thread can not share a cache line with blocks from other threads, otherwise there is false sharing once again, hence, the block size was rounded up to be a multiple of the cache line size (64 bytes), we ensure this happens by using `get_allocation_size`.

Combined `posix_memalign`, which aligns the newly allocated array to start at an address which is a multiple of the cache line size, this guarantees that our arrays start at the very beginning of a cache line, and consequently, every thread block within them will also start at the beginning of a cache line, eliminating false sharing.

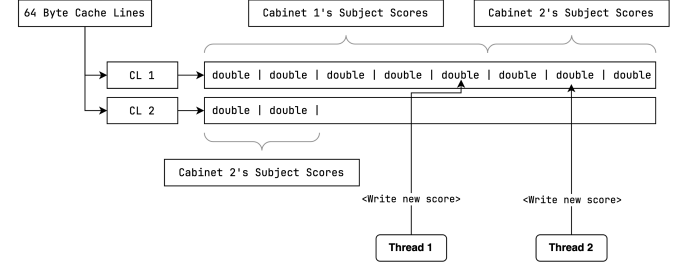


Figure 1: False sharing between two threads.

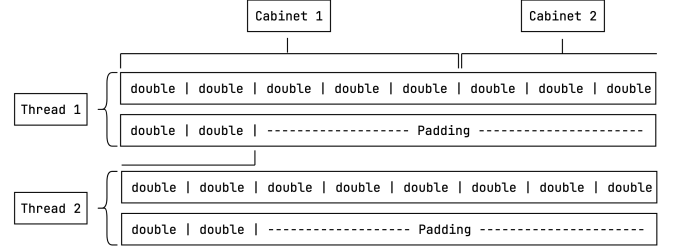


Figure 2: Per-thread new_score block.

In `recalculate_scores` the loop is over cabinets instead of documents. The `#pragma omp for` splits the C cabinets across T threads. Each thread takes a chunk of cabinets and for each one reads all T private blocks from `new_counts` and `new_scores`, sums them, and produces the centroid. There are no write conflicts since each thread writes to a disjoint set of blocks in `cabinets->scores`, in which each cabinet subject score block is padded and cache aligned to avoid false sharing. We also ensure we take advantage of spatial locality by assigning each thread a sequential

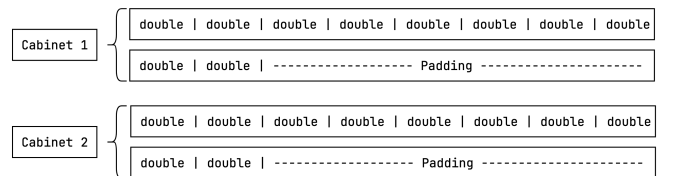


Figure 3: Padded cabinet scores.

The `swaps` variable is the only shared variable updated by all threads during reassignment. It counts how many documents changed cabinet in the current iteration and determines whether another iteration is needed. A `reduction(+:swaps)` clause gives each thread a private copy during the loop, avoiding data races, and sums all copies into the global `swaps` at the end.

5 Synchronization Concerns

Parallelization can lead to a number of problems which could cause the program to run in unexpected ways if not handled correctly. The root cause of unexpected behaviour lies to threads having to share data and perform on it concurrently, leading to race conditions. In order to eradicate incorrect and non-deterministic results, some synchronization was introduced, but avoided as much as possible, since it incurs sequential processing.

In order to avoid synchronization as much as possible, and as seen before, we created local thread storage for document counts and cabinet subject scores, meaning that each thread writes exclusively in its memory region. The main advantage of this approach is the unneededness of using `#pragma omp atomic` or `#pragma omp critical` in areas where data is operated on frequently. The disadvantage being that way more memory is used due to having to have storage per-thread, which is additionally incremented by having to pad the same memory blocks.

Nonetheless, even with the above mentioned efforts, synchronization is still required in specific areas. These are through deliberate implicit barriers from `#pragma omp for` directive, which encompasses the entirety of the code inside each of our three main algorithm functions. The reason as to why is due to subsequent processing requiring the previous computations.

```
assign_to_cabinets();
do {
    recalculate_scores();
    reassign_documents();
} while (<condition>);
```

The reason why the implicit barriers were left is due to:

- `assign_to_cabinets` - Every document needing to be assigned before calculating the scores of each cabinet.
- `recalculate_scores` - Every cabinet needing to have its score calculated before we try and reassign the documents.
- `reassign_documents` - Every document needing to be reassigned in order to either end processing or calculate the new cabinet scores.

Moreover, a `#pragma omp single` with an implicit barrier was used at the very beginning of the `reassign_documents` method. A single thread will reset the `swaps` variable and every thread will only start reassignment after that. This was done to ensure that no thread i resets the variable after all other $T - 1$ threads have finished reassignment, which might lead to incorrect results.

Overall, the strategies covered in this section minimises synchronization concerns as well as making sure processing will only start after all threads have finished their work.

6 Load Balancing

A consideration that is important whilst parallelizing an application is to making the workload spread as even as possible in order to avoid threads idling.

Therefore, in our solution this consideration came more prominently when dealing the work distribution in iterative tasks, more specifically `#pragma omp for` directive. We set an explicit clause of `schedule(static)` to divide the the iterations as evenly as possible.

More specifically for the reason behind the static scheduling can be observed at a per iteration cost. Requiring approximately the same quantity of calculations. As an example, to be able to obtain the the cabinet closest to a document we would calculate the distance for each cabinet. Therefore, given this uniformity that extends to the rest of the loops, the static scheduling made more sense as opposed to dynamic scheduling.

Overall, due to the even distribution of work per thread guaranteed that the idle periods of threads were minimised and negligible.

7 Performance Results

All measurements were collected on machine `lab6p6` using three input configurations: `ex1000` ($C = 1000$, $D = 50\,000$, $S = 200$), `ex10` ($C = 10$, $D = 200\,000$, $S = 100$), and `ex200` ($C = 200$, $D = 400\,000$, $S = 10$). For each configuration, the OpenMP implementation was executed ten times with $T \in \{1, 2, 4, 6\}$ threads. Speedup was computed relative to the one-thread OpenMP execution, while the serial implementation was measured separately to assess the overhead introduced by the OpenMP version itself.

Table 1: Run times and speedups for each input configuration.

Test	T	Mean (s)	Speedup
<code>ex1000</code>	Serial	203.6	—
	1	204.1	1.00×
	2	103.8	1.97×
	4	52.5	3.89×
	6	35.6	5.73×
<code>ex10</code>	Serial	235.6	—
	1	264.2	1.00×
	2	135.4	1.95×
	4	70.3	3.76×
	6	52.0	5.08×
<code>ex200</code>	Serial	246.1	—
	1	251.1	1.00×
	2	131.3	1.91×
	4	66.5	3.78×
	6	46.2	5.43×

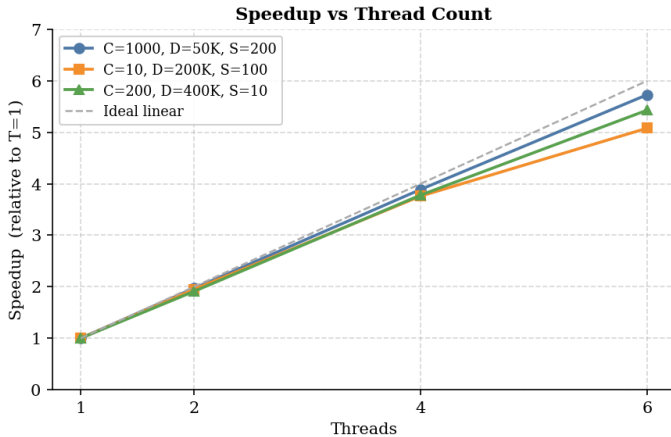


Figure 4: Observed speedup as thread count increased.

The results show near-linear scaling up to $T = 4$ across all three configurations. For `ex1000`, speedup increases from 1.97× at two threads to 3.88× at four threads and reaches 5.72× at six threads. `ex10` follows the same overall trend, reaching 1.95×, 3.76×, and 5.08× respectively. `ex200` shows similar behaviour, with speedups of 1.91×, 3.78×, and 5.43×. Across all tests, the largest deviation appears at six threads, where variability increases slightly, particularly in `ex200`. Compared to the serial version the one-thread OpenMP execution introduces little overhead in `ex1000`, moderate overhead in `ex200`, and a clearly larger overhead in `ex10`.

The stronger scaling observed in `ex1000` and `ex200` indicates that most execution time is spent in the document-parallel phases, where the workload is large enough to keep all threads occupied with little imbalance. In `ex10`, although the document count remains high, the cabinet count is very small, so the `recalculate_scores` phase offers limited parallel work and the fixed cost of synchronization becomes proportionally more visible. This effect is amplified at six threads, where barriers, reduction of thread-local accumulators, and the final merge of partial results contribute a larger fraction of total execution time. The elevated variability observed in `ex200` at six threads might be caused due to system interference since it is a shared machine.

8 Conclusion

The implementation described in this report with the respective results permits us to conclude that we have successfully parallelized the problem at hand. This has been achieved through careful yet structured implementation of local thread storage, minimization of synchronization, and work distribution. By balancing these different elements we have successfully achieved an efficient parallelized solution. The performance results confirm this, showing near-linear scaling up to four threads and speedups above 5× at six threads for all tested configurations. The remaining deviation from ideal scaling is consistent with synchronization overhead and with phases where the available parallel work becomes smaller, particularly in configurations with fewer cabinets.